Cybersecurity - Progetto Finale

Analisi della sicurezza di un'applicazione web Laravel attraverso l'individuazione di vulnerabilità critiche, la simulazione di attacchi reali e l'implementazione di mitigazioni efficaci secondo gli standard OWASP.

Made with GAMMA

Challenge 1: Assenza di Rate Limiting

La Vulnerabilità

Le rotte pubbliche GET dell'applicazione, come la visualizzazione degli articoli, non implementano alcun meccanismo di limitazione delle richieste. Questo espone il sistema ad attacchi Denial of Service (DoS), permettendo a un attaccante di inviare centinaia di richieste consecutive che sovraccaricano il server.

Senza rate limiting, qualsiasi utente può automatizzare richieste HTTP verso endpoint pubblici, causando rallentamenti significativi o il completo blocco del servizio.

Made with GAMMA

Challenge 1: Attacco e Mitigazione

Esecuzione Script → Server sovraccaricato → **Connection Refused**

L'attacco è stato simulato con uno script bash che invia 300 richieste parallele tramite **cURL**, mandando il sito offline con errore CONNECTION_REFUSED.

Soluzione Implementata

È stato integrato un Rate Limiter in Laravel che limita le richieste GET pubbliche a 30 al minuto per IP. Il middleware 'throttle:public-get' è stato applicato alle rotte critiche come /articles/show, /articles/index e /articles/search.

Risultato: il server risponde con HTTP 429 (Too Many Requests) bloccando l'attacco e mantenendo il servizio operativo.

Made with GAMMA

Challenge 2: Operazioni Critiche in GET

Cross-Site Request Forgery (CSRF)

L'applicazione espone **funzionalità critiche di assegnazione ruoli** tramite
L’applicazione espone operazioni critiche, come l’assegnazione dei ruoli, tramite richieste HTTP GET. Questo rende possibile un attacco CSRF: un attaccante può creare una pagina HTML malevola che, se visitata da un amministratore già autenticato, induce il browser a inviare automaticamente la richiesta sfruttando i cookie di sessione.
Il risultato è una **privilege escalation** verticale, in cui un utente non autorizzato ottiene privilegi amministrativi

Made with GAMMA

Challenge 3: Assenza di Logging

Principi Violati

Accountability: impossibile attribuire azioni agli utenti responsabili.

Non-repudiation: nessuna prova verificabile degli eventi di sicurezza.

In assenza di un sistema di logging adeguato, non è possibile ricostruire eventi di sicurezza né attribuire le azioni agli utenti responsabili, ad esempio dopo un attacco DoS o un cambio di ruolo non autorizzato.
Questo viola i principi di accountability e non-repudiation definiti dallo standard OWASP ASVS, che richiede il tracciamento di autenticazioni, modifiche ai dati e operazioni sensibili.
Senza log, l'applicazione non è in grado di rilevare, analizzare o dimostrare una violazione di sicurezza.

Mitigazione

La vulnerabilità è stata mitigata modificando il metodo HTTP delle rotte critiche da GET a PATCH. In questo modo, le operazioni di modifica dello stato non sono più eseguibili tramite richieste automatiche e sono protette dai meccanismi di sicurezza del framework.
Il tentativo di attacco CSRF genera ora una risposta HTTP 405 (Method Not Allowed), impedendo l'esecuzione dell'azione.

Made with GAMMA

Challenge 3: Implementazione dei Log

01

Autenticazione

Logging di login, logout e registrazione utenti tramite eventi Fortify. Ogni evento registra user_id e indirizzo IP.

02

Gestione Articoli

Tracciamento di creazione, modifica ed eliminazione articoli in ArticleController. I log includono user_id, article_id e IP.

03

Cambio Ruoli

Log di ogni assegnazione ruolo (admin, revisor, writer) con performed_by, target_user_id, new_role e IP. Utilizza Log::warning per evidenziare operazioni critiche.

Tutti gli eventi sono registrati in storage/logs/laravel.log, garantendo piena tracciabilità e conformità ai principi di accountability.

Made with GAMMA

Challenge 4: Server-Side Request Forgery

Attacco SSRF

L'applicazione consente agli utenti di selezionare API esterne per suggerimenti di notizie. Manipolando il menu a tendina tramite DevTools, un attaccante può forzare l'applicazione a effettuare richieste verso risorse interne non accessibili direttamente. In questo scenario, il server viene utilizzato come proxy per accedere a dati riservati, aggirando i controlli di accesso.

La vulnerabilità rientra nell'**OWASP Top 10** 2025 come Server-Side Request Forgery e consente l'esecuzione di richieste non autorizzate da parte del server stesso.

MITIGAZIONE:

L'SSRF è stato **mitigato** validando l'URL tramite whitelist e aggiungendo controlli di autorizzazione lato server, in modo che **solo utenti con privilegi adeguati possano effettuare richieste verso risorse interne**.

Made with GAMMA

Challenge 5 e 6: XSS e Mass Assignment

Cross-Site Scripting (XSS)

Durante la creazione di un articolo, il contenuto viene inizialmente protetto da auto-escape, impedendo l'esecuzione diretta di codice JavaScript. Intercettando e modificando la richiesta POST tramite Burp Suite, è stato possibile bypassare l'auto-escape e iniettare codice malevolo nel body dell'articolo. Il contenuto infetto viene salvato nel database ed eseguito ogni volta che l'articolo viene visualizzato, configurando uno stored XSS e consentendo, tra le altre cose, il furto dei cookie di sessione degli utenti.

Mitigazione

La vulnerabilità è stata mitigata introducendo un meccanismo di filtraggio del contenuto sia in fase di salvataggio che in fase di visualizzazione dell'articolo.
Nel controller `ArticleController` sono stati modificati i metodi `store` e `update` per sanitizzare il testo prima della persistenza nel database.
In aggiunta, la vista di visualizzazione dell'articolo è stata resa sicura applicando un rendering protetto, impedendo l'esecuzione di codice JavaScript eventualmente salvato.
In questo modo, anche gli articoli precedentemente infetti vengono mostrati come testo non eseguibile, bloccando definitivamente l'attacco stored XSS.

Mass Assignment Attack

A causa di una configurazione errata della proprietà `$fillable` nel modello User, l'applicazione accetta in massa anche campi non previsti dal form di profilo. Manipolando la richiesta tramite DevTools, un utente può inviare campi aggiuntivi che vengono automaticamente assegnati al modello, causando una privilege escalation.

Mitigazione

La vulnerabilità è stata mitigata limitando il `$fillable` esclusivamente ai campi effettivamente gestiti dal form, impedendo l'assegnazione diretta dei ruoli e proteggendo il modello da modifiche non autorizzate.

Made with GAMMA

Conclusioni e Risultati

6 Vulnerabilità Critiche

Identificate e documentate secondo standard OWASP

Attacchi Simulati

DoS, CSRF, SSRF, XSS e Mass Assignment eseguiti con successo

Mitigazioni Implementate

Rate limiting, conversione HTTP methods, logging completo, validazione input

Il progetto mostra come vulnerabilità comuni possano essere individuate e corrette tramite analisi applicativa e mitigazioni mirate, portando l'applicazione in linea con le best practice OWASP.

Made with GAMMA